

Tearframe 横纵分析报告

横纵分析法深度研究报告

研究时间：2026 年 6 月 | 所属领域：创作者工具 / 本地优先 AI 应用 | 研究对象类型：软件产品
(个人创作素材语料库)

作者: 数字生命卡兹克

研究时间：2026 年 6 月 | 所属领域：创作者工具 / 本地优先 AI 应用 | 研究对象类型：软件产品
(个人创作素材语料库)

一、一句话定义

Tearframe 是一个本地优先的「视频拉片语料库」，它不替你创作、也不替你思考，而是把你看过的每一条好视频拆成可填空的创作骨架，让外部 AI agent 通过一套契约把「好」结构化沉淀下来，越用越厚，最终变成你自己的创作肌肉记忆。

二、纵向分析：从一个痛点到一套契约

要看懂 Tearframe，得先看懂它在跟一个什么样的敌人较劲。这个敌人不是某个竞品，是一种很多创作者都熟悉的无力感。

你刷到一条特别好的视频。你知道它好。开头三秒就把你钉住了，中间有个反转让你「啊」了一声，结尾那句话你甚至想转发给朋友。你心里清楚，这条片的每一个环节都在精确地起作用。

然后呢？

然后你关掉它，第二天自己坐下来写脚本，那些「好」一点都带不走。你记得它好，但你说不清它好在哪个可复用的层面。下一条片你还是从零开始，凭手感，赌运气。

这就是 Tearframe 的起点。它的 `VISION.md` 把这个痛点写得很直白：

我看了很多好视频，知道它们好，但下次自己创作时，那些"好"无法复用。

整个项目，本质上是对这一句话的工程化回答。理解了这个出发点，后面所有看起来「过度设计」的地方——为什么要拆成八张卡、为什么要搞模板分级、为什么要做记忆图谱——都会变得顺理成章。它们都是在回答同一个问题：怎么让「好」可以被带走、被复用、被累积。

阶段一：把「拉片」从一份报告，重新定义成一副骨架

「拉片」这个词本身有历史。它是影视行业的老手艺，指对一部片子逐帧逐镜地拆解，研究它的叙事结构、镜头语言、剪辑节奏。传统拉片的工具，是 Premiere、DaVinci、MPC-HC 这类播放器加编辑器——你把视频拖到时间线上，一帧一帧地看，一边看一边在本子上记。后来有了浏览器插件（比如小镜故事板那一类），让你在网页里边播边截图加批注，最后导出一份文档。

但所有这些工具，产出的都是同一种东西：一份分析报告。

Tearframe 在这里做了第一个、也是最关键的一个观念切换。它的「信念一」写着：

拉片产物不是“分析报告”，是“可填空的骨架”。

这个区别听起来微妙，落到产品上却是天壤之别。一份报告是写给「过去那条片」的，你读完知道了它为什么好，然后报告就躺在文件夹里再也不会被打开。一副骨架是写给「未来你要拍的那条片」的——它抽掉了具体内容，只留下结构和句式，你下次可以直接往里填新东西。

报告是终点，骨架是起点。这一个观念上的偏移，决定了 Tearframe 后面所有的数据结构都不是为「记录」服务，而是为「复用」服务。它不是一个更好的拉片笔记本，它想做的是另一种东西。

阶段二：八维度框架——把一团模糊的「好」切成八刀

光说要做骨架还不够。一条视频的「好」是混在一起的，你得先把它切开，才能分别抽骨架。Tearframe 切了八刀，加两张高优视图，落成了它的核心 schema——八维度拉片框架。

这八张卡，每一张都对应一个具体的创作问题，而不是一个抽象的分析维度：

- **选题 (Topic)**：它在回答什么具体问题？为什么是此刻值得讲？
- **文案 (Copy)**：怎么把观点讲得不空？句式有什么共性？
- **结构 (Structure)**：用什么骨架？哪里加压，哪里反转？
- **镜头 (Shot)**：A-roll 和 B-roll 各自承担什么功能？
- **剪辑 (Edit)**：节奏地图长什么样，哪里停顿哪里加速？
- **配乐 (Music)**：情绪曲线、入点出点、动机？
- **字幕 (Subtitle)**：全字幕还是关键词，怎么强调？
- **节奏 (Pace)**：全片的信息密度怎么起伏？
- **账号 (Account)**：这条片在向观众承诺什么，为什么值得关注？

外加两张从其它维度切出来、但提到主卡级别的高优视图：**开头钩子 (Hook)**（前 3-5 秒逐帧）和**结构骨架**。为什么单独拎出来？因为对短视频来说，前三秒的留人率是生死线，它太重要了，不能埋在结构卡里一笔带过。

这套框架里藏着一个很聪明的设计——**类型权重表**。不是每条片都要把八张卡填满。一个「个人观点」类视频，重点在文案、节奏、钩子；一个「过程型 Vlog」，重点在节奏、剪辑、字幕；电影类则启用一套增强 lens，深拉镜头美学、台词戏剧性、蒙太奇、配乐动机。agent 拉片时先判断类型，再决定哪些卡必填、哪些可简、哪些深挖。

这一步看似只是个分类表，实际上解决了一个很现实的问题：**避免无差别的全维度填表，把分析力气花在这类视频真正重要的地方**。它承认了一件事——所谓「好」是有侧重的，访谈片的好和小纪录片的好不是一回事。

阶段三：解耦——「Agent 友好，而不是 Agent 全能」

这是 Tearframe 架构上最有判断力的一个决定，也是它区别于市面上几乎所有同类产品的分水岭。

大部分「AI 视频分析工具」的做法是端到端：你把视频丢进去，它内部调一个模型，吐给你一份分析。模型、流程、产物全都焊死在产品里。这种做法的好处是开箱即用，坏处是——你被锁死在它当时选的那个模型和那套 prompt 上。模型迭代了，产品不跟，你就只能用着一个越来越过时的「智能」。

Tearframe 的「信念三」反过来：

系统提供的是契约和资源，不是端到端流程。

它把自己切成了三层，而且层与层之间是硬解耦的：

- **UI 层**：纯数据驱动渲染。`VISION.md` 里有一句话定调子——「UI 永不调用 LLM，永不解析视频。UI 是渲染器，不是创作者。」所有「智能」都不发生在这一层。
- **服务层**：业务逻辑、存储、预处理流水线、对外的 MCP 协议。它是数据契约的执行者，只关心「输入是不是合法样片、输出是不是合法产物」。
- **拉片层**：通过一份 Skill 文档，让任意外部 agent 接入。今天用 Claude，明天用 GPT-5，后天用本地小模型，系统一行代码都不用改。

这个决定的代价，是 Tearframe 自己不「聪明」。它不分析视频，它把分析这件最难、最值钱、也最容易过时的事，外包给了 agent。它赌的是：**模型会越来越强，而契约可以不变**。把易变的（模型能力）和稳定的（数据 schema）分开，让稳定的那部分沉淀价值。这是一个典型的「以不变应万变」的工程哲学，也是这个项目最像一个「有想法的人做出来的东西」而非「又一个 AI 套壳」的地方。

从代码里能看到这套解耦不是 PPT。`packages/shared` 放共享的 Zod schema 和类型，前后端共用；`packages/server` 是 Express 后端，里面有 `SourceService`、`PreprocessService`、`TeardownService`、`MemoryService`、`GraphBuilder` 等一整套服务，还有一个独立的 `mcp/` 目录暴露 MCP 工具；`packages/web` 是 React 前端；`packages/skill` 是给外部 agent 看的协议说明书。四个包，职责泾渭分明。

阶段四：来源策略的一次关键转向——从 yt-dlp 到 OpenCLI

Tearframe 的演进里有一个清晰可见的「战略转向」，记录在 `OPENCLI_INTEGRATION.md` 和实施文档的 v1.1 更新里。

最早，样片抓取用的是 `yt-dlp`。这是个很自然的起手式——它是开源世界里下载视频的事实标准。但在中国创作者的真实场景里，`yt-dlp` 撞上了三堵墙：B 站、小红书、抖音的反爬需要手工导出 cookies，频繁失效；这些平台的支持本来就不稳定甚至不支持；而且 `yt-dlp` 只能拿到视频本体，拿不到评论、作者其他作品、官方摘要这些做拉片很需要的多维数据。

于是 Tearframe 把主通道换成了 **OpenCLI**——一个用「你已登录的 Chrome 会话」去操作平台的工具，零 cookies 配置。这一换，解决的不只是下载问题：

- B 站能直接拿**官方字幕**和**官方 AI 摘要**，跳过 Whisper；
- 小宇宙能拿**官方逐字稿**，这是拉播客的杀手锏，质量远超语音转写；
- 出错走 `sys.exit` 标准退出码（66 空结果 / 69 浏览器桥未连 / 77 需要登录 / 78 配置错误），后端能精确映射成友好的用户提示。

但 Tearframe 没有把 `yt-dlp` 一脚踢开。它的最终策略是「OpenCLI 主 + `yt-dlp` 兜底（仅 YouTube）」。

这是个克制的决定——YouTube 这块 OpenCLI 还没覆盖，那就让老将继续守着。这种「主力换新、老兵留岗」的姿态，比那种推倒重来的迁移更显成熟。

值得注意的是，Tearframe 在这里又一次贯彻了解耦原则。它的 `AGENTS.md` 和 `SKILL.md` 反复强调：外部 agent 不许直接调 `opencli`、`yt-dlp`、`ffmpeg`、`PySceneDetect`、`Whisper`。这些都是 `source.crawl`、`sample.import`、`sample.preprocess` 背后的实现细节。agent 只跟 Tearframe 的工具契约打交道，底层用什么抓取器是后端的事。哪天 OpenCLI 又被一个更好的东西取代，agent 侧的拉片流程一个字都不用动。

阶段五：资源复用模型——把「贵」的东西变成资产

拉片里有一类操作是「昂贵但可复用」的：镜头切分（`PySceneDetect`）、字幕抽取（官方字幕或 `Whisper`）、关键帧抽取（`ffmpeg`）。这些算一次要花真金白银的算力和时间。

Tearframe 的「核心创新」之一，是把这些预处理资源当成样片的**固有资产**，永久持久化在 `~/tearframe/samples/{id}/resources/` 下。它的逻辑是这样的：

agent 拿到拉片任务，先调 `sample.get_resources` 查一遍有没有现成的。镜头切过了？直接拿。没切？触发预处理，完成后回写。字幕、关键帧同理。这意味着——同一条样片可以拉 N 次，不同 agent 拉、不同 lens 拉、你成长之后再拉一次，但那份预处理资源**只生成一次，所有拉片共用**。

更精巧的是它给了「双重预处理路径」：Path A 让系统内置流水线做（推荐默认），Path B 让 agent 自带能力做完了再上传。两条路完成后产物**完全等价**，下次都能直接复用。这又是解耦思想的延续——系统不在乎资源是谁生成的，只在乎它合不合规、能不能复用。

阶段六：长片闸门——一道被「踩过坑」才会加的护栏

项目里有一处细节，几乎可以肯定是「出过事」之后补上的。

`AGENTS.md` 第 9 条和 `SKILL.md` 用很大篇幅规定了一件事：**任何超过 2400 秒（约 40 分钟）的视频，以及任何商业电影、纪录片、整集播客、整集剧集，禁止直接 `sample.import` 然后 `teardown.start`**。服务端会硬性拦截，返回 `LONG_FORM_TEARDOWN_BLOCKED`。

为什么？文档自己给了答案：

storyboard 校验要求覆盖每一个 shot，长片 shot 数动辄过千，agent 无法逐帧解读；8 维度卡片不适配商业长片叙事，单条样片挂太多 teardown 会丢失可复用价值。

解法是引入一个 **Collection 容器**：先 `collection.create` 建一个电影容器，`collection.import_master` 把整片软链进来（不复制、不降采样，只当容器），再 `collection.add_clip` 用 ffmpeg 切出一段独立的 ≤1080p 片段，对这个 clip 走标准拉片流程。clip 的时间码从 0 开始，UI 展示时再通过 `clip_start_sec` 换算回电影绝对时间，所有现有的 validator 一行都不用改。

这道闸门很能说明 Tearframe 的产品成熟度。它不是一个只跑过 demo 的玩具——从 output 目录里能看到 `walter_mitty_bilibili`（白日梦想家）、几百帧的 contact sheets、storyboard observations 分批文件，说明真有人拿这套流程去拉过一部电影的片段，并且在过程中撞到了「长片不能整片拉」这堵墙，于是回来给系统加了这道护栏。这种「被真实使用反推出来的约束」，是设计文档凭空想不出来的。

阶段七：质量闸门——跟「假完成」死磕

如果说前面几个阶段是在搭骨架，那么这个阶段是 Tearframe 投入心力最多、也最能看出它价值观的地方：它在跟 AI 拉片的「假完成」死磕。

什么是假完成？当你让一个 agent 去逐帧解读几百个镜头，它很容易偷懒——用脚本按 `phase × index % N` 这种规律批量填 `visual_summary`，或者在没真看关键帧的情况下，根据相邻镜头和字幕「推断」画面内容，凑出一份「292/292 镜头已解读」的漂亮报告。覆盖率 100%，内容全是水。

Tearframe 对这件事的态度近乎严厉。SKILL.md 里专门有一节叫「视觉断言与回溯条款」，把 `visual_summary`、`composition_analysis`、`camera_angle`、`shot_size` 这四个字段定义为「我看过这一帧」的**事实声明**。然后立下规矩：

如果当前 agent 环境拿不到关键帧的视觉内容……立刻停下来，把字段填成 `pending_visual_review`，并告诉用户这条样片需要在能看图的会话里二次过帧。绝不把模板化占位文本作为“已完成的拉片”提交。

而且它不是只靠 prompt 自觉。它有牙齿——`scripts/validate_storyboard.py`（16.5KB）、`validate_storyline.py`（16.7KB）这两个校验器会真的拦截批量重复的画面描述、混合景别、泛化机位、模板化构图、固定尾缀（「对应第 N 镜的具体落位」这类程序化痕迹）。`make_contact_sheets.py` 则负责把镜头拼成 12 格一张的大图证据，逼着 agent（和人）真的去看。

这件事很反直觉。一般产品恨不得让 AI 显得无所不能，Tearframe 反过来，专门设计机制去**揭穿 AI 的偷懒**，宁可让它诚实地说「这段我没看，需要补」，也不要一份华丽的假报告。这背后的信念是：拉片库的价值建立在每一条数据的真实性上，一条假数据污染的不仅是一条片，是整个会越用越厚的语料库。

阶段八：记忆图谱——从「单条拉片」到「越用越值钱」

最后一层，是 Tearframe 真正的野心所在，也是它区别于「一次性分析工具」的根本。

它的「信念二」写着：

单条样片的拉片产物 < 100 条样片聚合后的模板库。

单条拉片只是原料。真正的价值在聚合：第十次拉片，你能看到「反共识钩子」这个模板下已经有 7 个变体；第一百次拉片，你能看到「@某作者」的创作 DNA 长什么样。系统的价值是**指数增长的**，因为聚合视图来自所有历史拉片。

`MEMORY_GRAPH_DESIGN.md` 把这套记忆系统设计得相当完整。它特意区分了「事实源」和「派生索引」——Tearframe 自己的 SQLite 业务库（samples / teardowns / cards / storyboards / templates / relations）永远是唯一事实源；Graphiti（一个时间感知的知识图谱）只是派生索引后端，Graphiti 挂了，本地记忆层照常运行。这又是一个克制而清醒的边界划分：**不让一个外部依赖变成系统的命门**。

派生出来的记忆表包括：`memory_items`（每张卡、每个分镜、每个模板拆成的记忆条目）、`memory_relations`（跨拉片的相似关系）、`sample_scores`（各维度评分、置信度、理由）、`memory_clusters`（像「反常识钩子」「地点蒙太奇」「低成本采访结构」这样的聚类）。从代码侧看，`MemoryService.ts` 有 58KB，是整个后端最重的一个服务文件，说明这一层不是概念，是真写出来了。

模板沉淀也分了级：L1 是单条样片抽出的具体骨架（带来源 `sample_id`），≥3 个相似骨架聚合成 L2 模板族，再升华成跟选题角度无关的 L3 元模板。配合「作者风格 DNA」——聚合某作者所有拉片，自动算出他的选题偏好、钩子类型分布、结构偏爱、节奏指纹（信息密度曲线均值）。

这是整个项目的终态愿景：一年后，200+ 条精选样片、1500+ 条卡片、50+ 个高频复用的元模板，你开新选题先从模板库里找骨架再填内容，你能清楚地说出「我做的内容像 X 的钩子 + Y 的结构 + Z 的节奏」。

到这里，纵向的故事就完整了。从「我知道它好但带不走」这一句无力的抱怨开始，Tearframe 一步步搭出了一套把「好」结构化、可复用、可累积的系统：八维度切开它，解耦架构让任意 agent 来拆它，资源复用让昂贵的预处理只做一次，质量闸门保证拆出来的是真东西，记忆图谱让所有拆解汇成一座越用越厚的语料库。每一个阶段，都是在回答同一句开场白。

三、横向分析：竞争图谱

把 Tearframe 放到当下的工具版图里，会发现一个有意思的现象——它几乎没有正面竞品。不是因为太强，而是因为大部分相邻的工具，跟它根本不在解同一道题。

这属于横向分析里的「场景 A 偏 B」：直接对标物极少，但有一圈「看起来像、其实各走各路」的相邻物种值得逐一拆开看。我把它分成五类，每一类都活成了不一样的样子，而 Tearframe 恰好踩在它们都没占住的那块空地上。

3.1 第一类：传统拉片工具——手艺人的放大镜

代表选手：Premiere Pro、DaVinci Resolve、After Effects、Final Cut Pro、MPC-HC、VirtualDub、FFmpeg。

这是「拉片」这个词最原始的归宿。它们的共同点是：**给你一个极其精细的放大镜，但不给你任何结构**。你能逐帧暂停、放大、拖时间线、提取帧，看清每一个细节。但看清之后呢？所有的「理解」都发生在你的脑子里和你的笔记本上，工具不沉淀任何东西。

用户实际怎么用它们？影视专业的学生和从业者会一边用 MPC-HC 逐帧看，一边在 Word 或 Notion 里手敲分析。这个过程的产物是一份私人笔记，写完就锁进文件夹。它的优点是精度无上限，缺点是**零结构化、零复用、零累积**——你拉一百部片，得到的是一百份互不相干的笔记，不会自动长出任何模板。

Tearframe 跟它们的关系不是竞争，是「接管了它们产物的下游」。Tearframe 后端照样用 ffmpeg 抽帧、用 PySceneDetect 切镜头，但它把这些工具的输出喂进了一套 schema，让「看清」之后能「沉淀」。它解决的是这些工具压根不碰的那一半问题。

3.2 第二类：浏览器拉片插件——轻量但断头

代表选手：小镜故事板的 Chrome 拉片插件，以及一类「边看在线视频边截图加批注最后导出文档」的插件。

这一类比传统编辑器轻盈得多。直接用 Chrome 打开在线视频就能拉片，播放时一键截屏，插件自带注释字段，最后自动排版导出。对于「我就想快速记一下这条片的几个关键点」的需求，它很顺手。

但它的天花板也很明显：**它的终点还是一份导出的文档**。截图 + 批注 + 排版导出，本质上是把传统拉片笔记搬到了浏览器里，做得更快、更顺，但没有改变「产物是报告」这个底层范式。它不拆成结构化的维度，不抽可填空的骨架，更不会把你历次拉片聚合起来。

用户口碑里，大家夸的是「直接在浏览器里就能拉，不用下载视频」「截图批注一条龙」。这些夸奖恰恰暴露了它的定位——它是个**更顺手的笔记工具**，不是创作素材库。Tearframe 跟它的分野在于：插件让你把这一条片记得更快，Tearframe 让你把一百条片用得久。

3.3 第三类：短剧拉片 SaaS——云端的、封闭的、卖分析的

代表选手：量子探险这类「短剧拉片网站」。

这是相邻物种里最像 Tearframe 的一个，也最值得展开。它们主打「多维度分析引擎」，从剧情、镜头、节奏多个维度拆解短剧，号称有 5000+ 部短剧案例库、智能标签系统、专利算法。听起来跟 Tearframe 的八维度 + 模板库有几分神似。

但拆开看，三个根本差异：

第一，云端 vs 本地。 短剧拉片网站是 SaaS，你的样片、分析、数据都在它的服务器上。Tearframe 是 `local-first` —— `VISION.md` 明确「不做云端同步，本地优先，可备份目录」，所有数据躺在你自己的 `~/.tearframe/` 下。对一个把创作素材当作核心资产的创作者来说，这是「我的语料库到底归谁」的区别。

第二，封闭引擎 vs 开放契约。 SaaS 的分析引擎是黑盒，它用什么模型、怎么拆，你不知道也改不了，更换不了。Tearframe 把分析外包给你自己的 agent，模型可换、prompt 可调、流程可定制。SaaS 卖的是「我帮你分析好的结果」，Tearframe 给的是「你自己分析的能力和场地」。

第三，行业数据库 vs 个人语料库。 短剧网站的 5000+ 案例是平台的公共资产，服务的是「了解这个行业在流行什么」。Tearframe 的库是你一条条亲手收进来的私人精选，服务的是「形成我自己的创作 DNA」。一个是行业雷达，一个是个人肌肉记忆。

我的判断是：这一类 SaaS 和 Tearframe 服务的根本不是同一种人。SaaS 服务的是「想快速摸清短剧套路、批量做号」的团队；Tearframe 服务的是「想长期打磨自己创作品味、把好东西内化成自己语言」的独立创作者。前者要的是规模和速度，后者要的是主权和沉淀。

3.4 第四类：短视频采集分析软件 / 爆款预测 App——盯数据，不拆手艺

代表选手：各类「短视频内容分析采集管理软件」、Viral App、Viral Video、ViralShort 这类工具。

这一类的关键词是**数据**和**预测**，不是**手艺**。采集类软件帮你跟踪某个播主下所有视频，记录上传时间、点赞、评论、分享数，把视频元数据库化管理，方便对比哪条火了。爆款预测类（Viral App）让你上传自己还没发的视频，给一个 AI 表现预测和改进建议。

它们都在视频的「外部信号」上做文章——播放量、互动率、发布时机、这条会不会火。它们不关心、也拆不出「这条片的开头钩子用了哪种留人逻辑」「它的结构骨架是怎么加压反转的」这些**内部手艺**。

Tearframe 恰好相反，它对外部数据几乎不感兴趣（metrics 只是样片的一个附属字段），它全部的力气都花在拆内部手艺上。这两类工具回答的是完全不同的问题：采集/预测类回答「什么内容会火」，Tearframe 回答「这个让人停下来看完的效果，是怎么用钩子、结构、节奏、配乐配合做出来的，我怎么照着复刻一条」。一个面向运营和选品，一个面向创作和手艺。

3.5 第五类：AI 一站式视频创作软件——生成下游，拉片上游

代表选手：万兴喵影 2026（海外版 Filmora V15）、各类 AIGC 视频生成平台。

这一类是 2026 年最热的方向——把 AI 能力前置到「灵感触发第一秒」，输入文案就能得到结构化脚本和视频初稿，文生视频、图生视频一键生成素材，AI 擦除、抠像、超清、动感字幕一条龙，构建「构思—生成—润色—输出」的全链路。

它们和 Tearframe 看起来都在「帮创作者做视频」，但处在**价值链的两端**。AI 创作软件是**下游**——帮你把脑子里的想法快速变成成片。Tearframe 是**上游**——帮你把脑子里的「好的标准」先建立起来。

更有意思的是，它们其实是互补的。Tearframe 的终态愿景里，你开新选题时「先从模板库找骨架，再填具体内容」。那个填好的骨架和脚本，完全可以喂给 Filmora 这类工具去生成。Tearframe 负责「你该拍一条什么样的、用什么结构的片」，生成工具负责「把它快速做出来」。一个管品味和方法论，一个管效率和产能。

不过这里也藏着 Tearframe 最大的外部风险：如果 AI 创作软件强到可以「输入一条参考视频，自动学会它的风格并生成同款」，那「先拆解再复用」这个中间环节会不会被直接吸收掉？这个问题留到交汇洞察里再谈。

3.6 生态位小结

把这五类铺开看，Tearframe 占的是一块很清晰、也很少有人占的空地：

维度	传统编辑器	浏览器插件	短剧拉片 SaaS	采集/预测 App	AI 创作软件	Tearframe
核心产物	私人笔记	导出文档	云端分析报告	数据看板/评分	成片	可填空骨架 + 语料库
数据归属	本地	本地	云端（平台）	云端（平台）	云端为主	本地优先
智能来源	人脑	人脑	封闭引擎	封闭引擎	内置模型	可换的外部 agent
是否累积	否	否	平台累积	平台累积	否	个人指数累积
服务对象	影视从业者	快速记录者	做号团队	运营/选品	量产创作者	打磨品味的独立创作者
解决的问题	看清细节	快速记录	摸行业套路	预测会不会火	快速出片	让"好"可复用可累积

这张表里，Tearframe 那一列几乎没有跟任何一列重合。它不是某个工具的「更好版本」，它是一个被前面所有工具都漏掉的物种：**面向个人、本地优先、agent 解耦、以复用和累积为目的的创作素材语料库**。

当前这个赛道的格局，与其说是「百花齐放」或「两强争霸」，不如说是「各扫门前雪」——每一类工具都解决创作链路上的一段，没有谁站在「帮创作者把看过的好东西内化成自己创作语言」这个位置上。Tearframe 想站的，就是这块没人站的地。机会在于这块地确实空着且确实有人需要；风险在于，它空着也可能是因为——这个需求虽然真实，但愿意为之付出「认真拉片」这份功夫的人，本来就是极少数。

四、横纵交汇洞察

把纵向的发展脉络和横向的竞争图谱叠在一起看，几个一开始不明显的东西会浮出来。

第一个洞察：Tearframe 今天的「没有竞品」，是它纵向那几个观念决策亲手造成的。

它在横向上之所以跟谁都不正面撞，根子在纵向阶段一和阶段三那两次观念切换。「产物是骨架不是报告」让它跟所有以报告为终点的工具（编辑器、插件、SaaS）拉开了维度差；「Agent 友好而非全能」让它跟所有内置封闭引擎的工具（SaaS、预测 App、创作软件）走上了不同的技术路线。换句话说，它的「独占生态位」不是运气，是两个看似哲学、实则极其工程化的早期决策一路锁定出来的结果。这也解释了为什么它的对标物这么稀薄——能同时跨过这两道观念门槛的产品，本来就不多。

第二个洞察：如果把竞品也放上时间线，会发现一个分叉点。

短剧拉片 SaaS、采集 App、AI 创作软件，它们的演化路径几乎都指向同一个方向——**云端化、规模化、自动化**，把越来越多的判断收进平台的黑盒里，让用户越来越省心、也越来越依赖。这是过去十年 SaaS 的主流叙事。

Tearframe 的演化路径反着来——**本地化、个人化、解耦化**，把判断权、数据权、模型选择权统统交还给用户。它的纵向历程里，OpenCLI 用「你自己的 Chrome 会话」、记忆系统坚持「本地是事实源、Graphiti 只是派生」、架构坚持「智能在 agent 那端」，每一步都在往「用户主权」上加码。这正好踩在 2026 年「local-first + 个人 AI agent + MCP 协议」这股暗流上。它不是在跟那些 SaaS 抢同一批用户，它是在赌一种不同的范式会起来：未来的工具不是「替你做」，而是「给你能力和场地，你自己做」。这个赌注对不对，是它最大的不确定性，但方向上它站在了一个有故事的位置。

第三个洞察：它今天的每一个优势，都能在历史里找到那个埋下伏笔的节点。

- 优势「任意模型可换、不被锁死」——根在阶段三的三层解耦决策。
- 优势「拉播客/B 站能零成本拿高质量字幕」——根在阶段四从 yt-dlp 转向 OpenCLI 那次转向。
- 优势「同一条片可多次拉、预处理不重复花钱」——根在阶段五的资源复用模型。
- 优势「拉出来的是真东西不是水报告」——根在阶段七跟「假完成」死磕时立下的视觉断言条款和那两个十几 KB 的校验器。
- 优势「越用越值钱」——根在阶段八的记忆图谱和模板分级。

一个产品的优势能这样一条条回溯到具体的历史决策，说明它的演进是有内在逻辑的，不是功能的随意堆叠。

第四个洞察：它今天的劣势，恰恰是当初那些「好决策」的影子。

「Agent 友好而非全能」是个好决策，但它的代价是——**Tearframe 本身不开箱即用**。它需要你自己接一个 agent、配 OpenCLI、装 Chrome 扩展、登录各平台、跑 PySceneDetect 和 Whisper。**README** 里那一长串安装步骤就是证据。对一个普通创作者，这道门槛足以劝退。当初那个让它「不被模型锁死」的解耦决策，今天变成了「难以被普通人用起来」的包袱。

「本地优先、不做协作、不做云同步」也是个有性格的决策，但它把 Tearframe 钉在了「单机、单人」的天花板下。它注定长不成一个有网络效应的平台——你的语料库再厚，也只是你一个人的，无法变成社区共享的模板市场。当初那个保护「用户主权」的决策，今天框住了它的「想象空间」。

还有阶段七那套近乎严苛的质量闸门。它保证了数据真实，但也意味着——**认真用 Tearframe 是一件很费功夫的事**。它逼着你（和你的 agent）真的去逐帧看、不许偷懒。这跟横向那些「上传视频一键出分析」的工具形成了刺眼的反差。它的高质量，本身就是它的高门槛。当初那个守护语料库纯净度的决策，今天劝退了想图省事的大多数。

这是所有有判断力的产品都会遇到的宿命：让你与众不同的那个决策，往往同时就是限制你长大的那个决策。Tearframe 的每一个长处，背面都贴着一个对应的短处，而且它们是同一枚硬币。

未来三个剧本。

最可能的剧本：Tearframe 留在「极客创作者的私人神器」这个位置。它不会爆发式增长，但会有一小撮认真打磨内容的独立创作者把它用成主力——他们愿意付出配置和拉片的功夫，因为他们真的在乎自己的创作语言能不能沉淀。它像一把好用但需要养护的手工刀，不会进千家万户，但用的人离不开。

最危险的剧本：AI 创作软件向上游吃。当 Filmora 这类工具强到「丢一条参考视频，自动学风格并生成同款」，「先拆解、抽骨架、再复用」这个 Tearframe 赖以存在的中间环节被端到端的生成能力直接吞掉。用户不再需要理解一条片为什么好，反正 AI 能直接帮我做一条「像它一样好」的。到那天，Tearframe 的「让好可复用」会显得多此一举——因为复用被自动化了，理解变得不必要。这是它最深的生存焦虑：它押注的是「创作者想理解并内化好东西」，而生成式 AI 押注的是「创作者根本不需要理解，要结果就行」。

最乐观的剧本：local-first + 个人 agent 这股范式真的起来了，MCP 成了 AI 时代的 HTTP，每个创作者都有一个常驻的、了解自己的 AI agent。这时候 Tearframe 的解耦架构突然变成了巨大的先发优势——它早就为「任意 agent 接入」准备好了契约，它本来就是为这个世界设计的。当配置门槛被一个足够聪明的 agent 自动抹平，它的所有「劣势」都消失，只剩下「越用越厚的个人创作语料库」这个别人补不上的护城河。在这个剧本里，它从「极客神器」长成了「每个认真创作者的标配第二大脑」。

哪个剧本会发生，取决于一个更大的问题——在 AI 越来越能直接给结果的时代，「理解」本身还值不值钱。

Tearframe 的整个存在，是对「值钱」这一边下的注。它相信创作者不只想要「一条好片」，更想要「成为一个能持续做出好片的人」。前者要的是产物，后者要的是肌肉记忆。这两件事的区别，跟「给你一条鱼」和「让你长出钓鱼的手感」是同一个区别。绕了一大圈，它最终回到了开头那句话——你

看了很多好视频，知道它们好。Tearframe 想做的，是让那个「知道」不再是一闪而过的感觉，而是能被你一次次调用的能力。

它赌的是：哪怕 AI 能替你做出好片，你仍然想知道「为什么好」，并把这份知道，变成你自己的。

五、信息来源

本报告基于对项目本地代码库的直接通读，以及对相邻工具市场的联网调研。

项目内部一手来源（`/Users/ryanbzhou/Developer/vibe-coding/freedom/tearframe`）：

- `README.md` — 项目定位、快速开始、质量闸门说明
- `AGENTS.md` — 强制拉片 workflow、长片 Collection 约束（第 9-10 条）
- `docs/VISION.md` — 三个核心信念、八维度框架、资源复用模型、模板沉淀、终态愿景
- `docs/OPENCLI_INTEGRATION.md` — 从 yt-dlp 到 OpenCLI 的转向动机、平台适配器、退出码映射、Skill 协同
- `docs/MEMORY_GRAPH_DESIGN.md` — 记忆图谱设计、事实源 vs 派生索引、Graphiti 集成
- `docs/IMPLEMENTATION.md` — 技术栈固化、仓库结构、依赖前置
- `docs/architecture.md` — 三层解耦总览
- `packages/skill/SKILL.md` — 拉片协议、详情页 Tabs 供料契约、视觉断言与回溯条款
- `packages/server/src/db/schema.ts` — SQLite 业务表与派生记忆表结构
- `packages/server/src/mcp/tools.ts` — MCP 工具契约定义
- 源码服务文件规模佐证（`MemoryService.ts` 58KB、`GraphBuilder.ts` 19.5KB、`TeardownPage.tsx` 60KB、两个 storyboard/storyline validator 各约 16KB）
- `output/` 目录下 `walter_mitty_bilibili`、contact sheets、storyboard observations 等真实拉片产物
- Git 提交历史（3 次提交，2026-05-29 至 2026-06-01）

横向竞品/同类调研（联网，访问时间 2026-06-02）：

- 知乎《拉片用什么软件来拉呢？》— 传统编辑器与小镜故事板插件 — <https://www.zhihu.com/question/29175621/answer/85826048575>
- 《用哪些软件拉片》— 逐帧分析工具清单 — <http://momobiotech.com/post/158983.html>
- CSDN《短剧拉片网站 2026 推荐》（量子探险多维度分析引擎、5000+ 案例库） — <https://blog.csdn.net/a1443989385/article/details/160829290>
- 华军软件园《短视频内容分析采集管理软件》— 数据库化采集与播主跟踪 — <https://www.onlinedown.net/soft/1229353.htm>

- 腾讯网《万兴喵影 2026 一站式 AI 视频创作闭环》— AI 全链路生成 — <https://new.qq.com/rain/a/20251201A086Q800>
- App Store《Viral App: 视频分析》— 爆款表现预测 — <https://apps.apple.com/cn/app/id6755832459>
- 小镜故事板官网 — AI 分镜创作定位 — <https://xjstoryboard.com/>
- 腾讯云开发者社区《MCP 在 Agent 架构中的位置》— MCP 协议作为工具调用层的背景 — <https://cloud.tencent.com.cn/developer/article/2613746>

部分横向竞品的精确营收、用户规模等商业数据公开渠道暂缺，本报告在相关处未做量化断言，仅基于公开产品定位做定性对比。

方法论说明

本报告采用横纵分析法（Horizontal-Vertical Analysis，由数字生命卡兹克提出）：纵轴追溯研究对象从诞生到当下的完整发展脉络与决策逻辑，横轴在当前时间截面上与同类/相邻工具做系统性横向对比，最后交叉两条轴产出综合判断与未来推演。